1 Computer Vision

1.1 Feature Extraction and Img-Proc for Computer Vision [NA20]

[15] There are nearly 100 million sensors dispersed around the retina. There are two types of sensors: the **rods** for black and white vision, and **cones** for color vision. 100 million cones and 10 million rods. One type of rod and three types of cones, to distinguish red, blue and green. The human eye can distinguish between 32 levels of gray [17].

1.1.1 Image formation

$N \times N$, m *bits* pixels: N controls the number of points and m controls the number of brightness values: 2^m values, ranging from 0 to $2^m - 1$. The ideal value of m is related to the signal-to-noise ratio of the camera[45]. High order bits carry more information, the least significant bit carries more information and it is noise[46]. Color images specify three pixel intensities. The color scheme can be RGB, CMYK, etc. To specify a pixel color it is possible to specify an index for each pixel. The index is referenced to a table called palette. Disadvantage: only a reduced collection of colors can be used, depending on the size of the palette[46]. Alternative: use several image planes to store the color components of each pixel (called true color).

1.1.1.1 The Fourier Transform [48]

Fourier transf of the signal p is: $Fp(\omega) = \int_{-\infty}^{+\infty} p(t) e^{-j\omega t} dt$ $\omega = 2\pi f$ [rad/s] $e^{-j\omega t} = \cos(\omega t) - j\sin(\omega t)$ $j = \sqrt{-1}$

The Fourier transform is a complex number, the real part is plotted. The Fourier transform is represented as magnitude and phase. $|Fp(\omega)| = \sqrt{\Re(Fp(\omega))^2 + \Im(Fp(\omega))^2}$ $arg(Fp(\omega)) = \tan^{-1}\left(\frac{\Im(Fp(\omega))}{\Re(Fp(\omega))}\right)$ The inverse Fourier transform of a signal exists. The phase carries the most information to reconstruct the image.

Convolution: $p_1(t) * p_2(t) = \int_{-\infty}^{\infty} p_1(\tau)\, p_2(t-\tau)\, d\tau$

$F(p_1(t) * p_2(t)) = Fp_2(t) \times Fp_1(t)$ the frequency dual of convolution is multiplication. Instead of computing the convolution in time domain, it is possible to multiply signals in the frequency domain and then execute the inverse Fourier transform.

Correlation $p_1(t) \circ p_2(t) = \int_{-\infty}^{\infty} p_1(\tau)\, p_2(t+\tau)\, d\tau$ gives a measure of the match between the two signals.

$delta(t-\tau) = 1$ *if* $t = \tau$, 0 *otherwise.*

Nyquist sampling criterion [55]: the sampling frequency must be at least twice the highest frequency of the sampled signal.

Discrete Fourier transform (DFT) $Fp_u = \frac{1}{N}\sum_{x=0}^{N-1} p_x e^{-j\frac{2\pi}{N}xu}$:there are N points p_x $x \in [0, N-1]$ u is the discrete frequency [59], sampled at the frequencies 0, f, 2f, etc. In the space, samples are equally spaced. The frequency is sampled at the reciprocal of the spacing in time/space. The inverse DFT is $p_x = \sum_{u=0}^{N-1} Fp_u e^{j\frac{2\pi}{N}ux}$. Assumptions prior to applicating the DFT: the sampling criterion must be satisfied (Nyquist frequency) and the sampled function must replicate to infinity.

Two dimensional transform[62] Pixels have coordinates (*x*,*y*), (*u*,*v*) are the two-dimensional horizontal and vertical frequencies. $FP_{u,v} = \frac{1}{N^2}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1} P_{x,y} e^{-j\frac{2\pi}{N}(ux+vy)}$ The 2D inverse DFT is $P_{x,y} = \sum_{u=0}^{N-1}\sum_{v=0}^{N-1} FP_{u,v} e^{j\frac{2\pi}{N}(ux+vy)}$.[64] It is common to use logarithms of the magnitude to display Fourier transforms. The Fast Fourier Transform can only be applied to square images whose size is an integer power of 2.

Discrete Cosine Transform (DCT)[75]: a fast DCT exists; DCT gives better compression of the same quality than FFT; JPEG and MPEG are based on DCT. The DCT is shift variant. The Discrete Sine Transform also exists, but it has less applications.

Discrete Hartley Transform(DHT)[77]

1.1.2 Image processing [91]

1.1.2.1 Image description (Histograms)

The intensity histogram plots: the frequency of each brightness level against the brightness level. The image contrast is measured by the range of brightness levels.

Point operations: each pixel value is replaced with a new value obtained from the old one. A linear transformation that modifies the brightness value of the point (x, y) from **Old** to **New** using the formula: $N_{x,y} = k \times O_{x,y} + l$. k : gain. l :level.

Form 1

Histogram normalization: the histogram is stretched, so given the transformation, the minimum intensity with non-zero frequency becomes the minimum brightness value (e.g.zero) and the maximum intensity with non zero frequency becomes the

maximum brightness value (e.g.255): It means solving the system to get a gain and level.
Histogram equalization: a non linear mapping from an input image to an output image. The output image is produced so that the brightness frequencies are constant. Each old level $O(l)$ is mapped to a new level $N(l)$. N_{img} : the number of pixels in the image and also the sum of all frequencies. $[L_{min}, L_{max}]$: the level range.

If the new image, for each level, has a constant frequency, then the constant frequency must be $N(l)=\frac{N_{img}}{L_{max}-L_{min}}$; the integral of the frequencies up to the brightness level q is $\sum_{l=0}^{q} N(l)=q*\frac{N_{img}}{L_{max}-L_{min}}$. The sum of the old-image frequencies, up to the level p , need the brightness levels $[0,q]$ of the new image. Note that in the new image, the brightness frequency is constant. Given the mapping, the new image takes q levels to map the old p brightness levels.
$\sum_{l=0}^{p} O(l)=\sum_{l=0}^{q} N(l)=q*\frac{N_{img}}{L_{max}-L_{min}}$ => $q=\frac{L_{max}-L_{min}}{N_{img}}*\sum_{l=0}^{p} O(l)$, so the brightness value p is mapped to q [98].
If we apply a linear transformation (Form 1) to an image and then histogram equalization, the result is always the same.
[98] Histogram equalization finds little use in generic image processing systems, except for display. Hist.Norm is preferred.
Thresholding[98] this operator selects pixels that have a value or are within a range. Two main forms: uniform and adaptive thresholding.
Optimal thresholding. The Otzu technique[100]: it maximizes the likelihood that the chosen threshold splits the image into object and background. The hypothesis is that the object has a different range of intensities from the background. It starts from frequency histogram, where each frequency is divided by the number of image points, for each level we calculate the probability that an image point have that brightness level: a probability distribution. $p(l)=\frac{N(l)}{N_{img}}$ We can calculate:

$\omega(k)=\sum_{l=1}^{k} p(l)$	Zero order cumulative moment. The probability that a point has a brightness less or equal than k , that is, in the range $[0,k]$
$\mu(k)=\sum_{l=1}^{k} l*p(l)$	First order cumulative moment: the expected brightness, over the brightness levels $1\le l\le k \quad l\in[1,k]$
$\mu_T=\sum_{l=1}^{L_{max}} l*p(l)$	Expected brightness level, over the entire image.
$\sigma_B^2(k)=\frac{(\mu_T*\omega(k)-\mu(k))^2}{\omega(k)*(k-\omega(k))}$	Variance of the class separability. The separability between the variables of the same class

The optimal threshold is $T_{opt}=argmax(\sigma_B^2(k)) \quad k\in[1,L_{max}]$ [101]

1.1.2.2 Group operations

Template convolution : the convolution notation is $N=W*O$, where W is the template of weighting coefficients[105]. [109]

2 Biography

ID	Title	Version
[NA20]	"Feature Extraction and Image Processing for Computer Vision", M.Nixon, A.Aguado. Book web site	4th Edition, 2020

3 Links

- Cvonline The Evolving, Distributed, Non-Proprietary, On-Line Compendium of Computer Vision